

Tristan Jones
11/3/25
CSCI 8110

Project 3: Implement SENet into VGG16

1) Overview

Essentially, this program is designed to load the pretrained VGG16 model with ImageNet weights. Then we implement our own attention module in this case being SENet. The SENet block is input into six different positions in the VGG16 architectures and we then train a baseline model with the SENet-augmented variant of the model using the Imagenette database. Then lastly we train again but only the attention and classifier layers train this time.

2) Code Structure

Part 1 Data Preperation

```
def build_datasets(root: str):
    """
    Build tf.data datasets from a folder structure like:
    root/train/{class}/image.jpg
    root/val/{class}/image.jpg
    """
    train_dir = os.path.join(root, "train")
    val_dir = os.path.join(root, "val")

    train_ds = tf.keras.preprocessing.image_dataset_from_directory(
        train_dir,
        labels="inferred",
        label_mode="categorical",
        batch_size=BATCH_SIZE,
        image_size=IMAGE_SIZE,
        shuffle=True,
        seed=1337
    )
    val_ds = tf.keras.preprocessing.image_dataset_from_directory(
        val_dir,
        labels="inferred",
        label_mode="categorical",
        batch_size=BATCH_SIZE,
        image_size=IMAGE_SIZE,
        shuffle=False,
        seed=1337
    )

    # Optional: cap dataset size for quick iteration
    if MAX_TRAIN_IMAGES is not None:
        train_ds = train_ds.unbatch().take(MAX_TRAIN_IMAGES).batch(BATCH_SIZE)
    if MAX_VAL_IMAGES is not None:
        val_ds = val_ds.unbatch().take(MAX_VAL_IMAGES).batch(BATCH_SIZE)

    # Preprocessing
    def _prep(x, y):
        return preprocess_input(tf.cast(x, tf.float32)), y

    train_ds = train_ds.map(_prep, num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    val_ds = val_ds.map(_prep, num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    return train_ds, val_ds
```

Essentially this loads the images with Keras' image dataset from directory resized from 224x224 and preprocessed with VGG16's preprocess input (This was later changed to 160x160 for speed)

Part 2 VGG16 backbone

```
def vgg16_backbone():  
    """  
    Load VGG16 conv body with pretrained weights.  
    include_top=False to allow a 10-class head for Imagenette.  
    """  
    base = VGG16(weights="imagenet", include_top=False, input_shape=IMAGE_SIZE + (3,))  
    for layer in base.layers:  
        layer.trainable = False # freeze VGG conv weights  
    return base
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102,764,544
fc2 (Dense)	(None, 4096)	16,781,312
predictions (Dense)	(None, 1000)	4,097,000

What this does is it freezes the convolutional layers for transfer learning. But it doesn't freeze the other layers so they can still be trained more.

Part 3 SENet Block

```
def se_block(input_tensor, ratio: int = 16, name: str = "se"):
    """
    Squeeze-and-Excitation block.
    - Squeeze: GlobalAveragePooling2D
    - Excitation: Dense -> ReLU -> Dense -> Sigmoid
    - Scale: multiply
    """
    channels = input_tensor.shape[-1]
    se = layers.GlobalAveragePooling2D(name=f"{name}_gap")(input_tensor)
    se = layers.Dense(max(channels // ratio, 1), activation="relu", name=f"{name}_fc1")(se)
    se = layers.Dense(channels, activation="sigmoid", name=f"{name}_fc2")(se)
    se = layers.Reshape((1, 1, channels), name=f"{name}_reshape")(se)
    x = layers.Multiply(name=f"{name}_scale")([input_tensor, se])
    return x
```

So for this Squeeze and Excitation block, the global average pooling squeezes spatial info to channel descriptors, then two FC layers produce channel-wise gates.

Essentially 3 steps:

Squeeze (GAP) -> Excitation (FC-ReLU-FC-Sigmoid) -> Scale (channel-wise multiply)

Part 4 New Model Assembly

```
def build_model_with_se(position: str, se_ratio: int = 16) -> tf.keras.Model:
    """
    Build a model that inserts SE at a specified position.
    Allowed positions:
    - 'before_conv1_1' (between input and conv1_1)
    - 'between_pool1_conv2_1'
    - 'between_pool2_conv3_1'
    - 'between_pool3_conv4_1'
    - 'between_pool4_conv5_1'
    - 'between_pool5_dense' (i.e., right before head)
    """
    allowed = {
        "before_conv1_1",
        "between_pool1_conv2_1",
        "between_pool2_conv3_1",
        "between_pool3_conv4_1",
        "between_pool4_conv5_1",
        "between_pool5_dense",
    }
    if position not in allowed:
        raise ValueError(f"position must be one of {allowed}, got {position}")

    base = vgg16_backbone()
    inputs = base.input

    # Get important tensors
    layer_dict = {l.name: l for l in base.layers}
    # pool outputs to tap
    pool1 = layer_dict["block1_pool"].output
    pool2 = layer_dict["block2_pool"].output
    pool3 = layer_dict["block3_pool"].output
    pool4 = layer_dict["block4_pool"].output
    pool5 = layer_dict["block5_pool"].output

    # Use the layer graph to continue from an insertion tensor through the rest of the base
    def continue_from(tensor, start_layer_name: str):
        """Feed 'tensor' into the base network starting at layer 'start_layer_name' inclusive."""
        take = False
        x = tensor
        for l in base.layers:
            if l.name == start_layer_name:
                take = True
            if take:
                x = l(x)
        return x
```

```

        return x

    if position == "before_conv1_1":
        x = se_block(inputs, ratio=se_ratio, name="se_at_input")
        # continue from first conv
        x = continue_from(x, "block1_conv1")
    elif position == "between_pool1_conv2_1":
        x = se_block(pool1, ratio=se_ratio, name="se_after_pool1")
        x = continue_from(x, "block2_conv1")
    elif position == "between_pool2_conv3_1":
        x = se_block(pool2, ratio=se_ratio, name="se_after_pool2")
        x = continue_from(x, "block3_conv1")
    elif position == "between_pool3_conv4_1":
        x = se_block(pool3, ratio=se_ratio, name="se_after_pool3")
        x = continue_from(x, "block4_conv1")
    elif position == "between_pool4_conv5_1":
        x = se_block(pool4, ratio=se_ratio, name="se_after_pool4")
        x = continue_from(x, "block5_conv1")
    elif position == "between_pool5_dense":
        # place after pool5, right before head
        x = se_block(pool5, ratio=se_ratio, name="se_after_pool5")
    else:
        raise RuntimeError("Unhandled position")

    outputs = build_classifier_head(x, num_classes=NUM_CLASSES, name="cls")
    model = models.Model(inputs=inputs, outputs=outputs, name=f"vgg16_se_{position}")
    # Freeze all VGG16 layers. SE + head are trainable by default.
    for l in model.layers:
        if l.name.startswith("block"):
            l.trainable = False
    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-3),
        loss="categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

```

SE Blocks are inserted at six different positions across the VGG16 body, which allows us to look at where reweighting of these channels is the most effective.

Part 5 Training and evaluation step

```
def train_and_eval(model: tf.keras.Model, train_ds, val_ds, tag: str):
    ckpt_path = os.path.join(OUTPUT_DIR, f"{tag}_best.keras")
    cbs = [
        tf.keras.callbacks.ModelCheckpoint(ckpt_path, monitor="val_accuracy",
                                           save_best_only=True, save_weights_only=False),
        tf.keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=3, restore_best_weights=True)
    ]
    history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, callbacks=cbs)
    eval_res = model.evaluate(val_ds, return_dict=True)
    # Save history & metrics
    # Use UTF-8 encoding when writing JSON to avoid platform-dependent encoding issues
    with open(os.path.join(OUTPUT_DIR, f"{tag}_history.json"), "w", encoding="utf-8") as f:
        json.dump({k: [float(x) for x in v] for k, v in history.history.items()}, f, indent=2)
    with open(os.path.join(OUTPUT_DIR, f"{tag}_metrics.json"), "w", encoding="utf-8") as f:
        json.dump({k: float(v) for k, v in eval_res.items()}, f, indent=2)
    print(f"[RESULT] {tag}: {eval_res}")
    return eval_res
```

Essentially, this section lays out exactly how the model handles training these new image weights. This compiles with Adam(1e-3), which is a stochastic gradient descent model. This also implements categorical crossentropy, accuracy, early stopping and model checkpoints all of which are relevant to what we want for this comparison. Afterward, we get some basic metrics and save them to a history and metrics .json file for graphing.

Part 6 Visualization Utilities

```
def visualize_feature_maps(model: tf.keras.Model, image_paths: List[str], layer_names: List[str], tag: str, max_channels: int = 16):
    """
    For each image, extract activations from specified layers and save montage images.
    """
    # Build sub-model to output selected activations
    outputs = [model.get_layer(name).output for name in layer_names]
    act_model = tf.keras.Model(inputs=model.input, outputs=outputs)

    for img_path in image_paths:
        img = tf.keras.utils.load_img(img_path, target_size=IMAGE_SIZE)
        x = tf.keras.utils.img_to_array(img)[None, ...]
        x = preprocess_input(x)

        activations = act_model.predict(x)
        base_name = os.path.splitext(os.path.basename(img_path))[0]

        for lname, act in zip(layer_names, activations):
            # act shape: (1, H, W, C)
            act = act[0]
            C = act.shape[-1]
            show_c = min(C, max_channels)
            cols = int(math.sqrt(show_c))
            rows = int(math.ceil(show_c / cols))

            fig = plt.figure(figsize=(cols * 2, rows * 2))
            for i in range(show_c):
                ax = fig.add_subplot(rows, cols, i + 1)
                fm = act[:, :, i]
                fm = (fm - fm.min()) / (fm.max() - fm.min() + 1e-8)
                ax.imshow(fm, cmap="viridis")
                ax.axis("off")
            fig.suptitle(f"{tag} - {lname} - {base_name}", y=0.95)
            out_path = os.path.join(OUTPUT_DIR, f"{tag}_{lname}_{base_name}.png")
            plt.savefig(out_path, bbox_inches="tight")
            plt.close(fig)

    print(f"[INFO] Saved feature map visualizations for tag={tag} to {OUTPUT_DIR}")
```

This section really just saves the feature map images and formats them in a specific way so its easily readable.

```
def save_before_after_bar(output_dir: str):
    """Create and save a grouped bar chart comparing baseline vs SE insertions.

    The function looks for <tag>_history.json files in output_dir and extracts the
    final validation accuracy for baseline and each `se_<position>` experiment.
    """
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    positions = [
        "before_conv1_1",
        "between_pool1_conv2_1",
        "between_pool2_conv3_1",
        "between_pool3_conv4_1",
        "between_pool4_conv5_1",
        "between_pool5_dense",
    ]

    def load_final_val(path: str):
        if not os.path.exists(path):
            return None
        with open(path, "r", encoding="utf-8") as f:
            j = json.load(f)
        val_acc = j.get("val_accuracy") or j.get("val_acc")
        if val_acc:
            return float(val_acc[-1])
        return None

    baseline_val = load_final_val(os.path.join(output_dir, "baseline_history.json"))
    se_vals = []
    for pos in positions:
        fname = os.path.join(output_dir, f"se_{pos}_history.json")
        v = load_final_val(fname)
        se_vals.append(v if v is not None else 0.0)

    # Create grouped bar chart
    x = np.arange(len(positions))
    width = 0.35
    plt.figure(figsize=(10, 6))
    baseline_list = [baseline_val] * len(positions)
    plt.bar(x - width / 2, baseline_list, width, label="baseline (before)", color="tab:blue", alpha=0.9)
    plt.bar(x + width / 2, se_vals, width, label="with SE (after)", color="tab:orange", alpha=0.9)

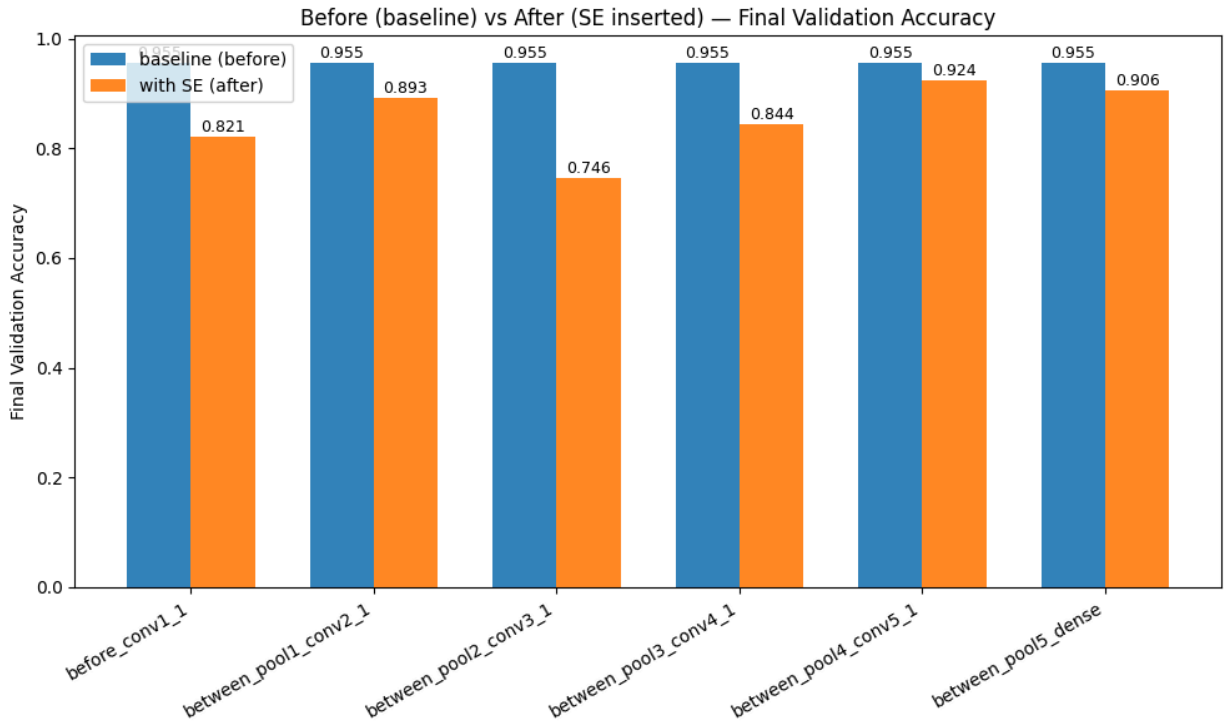
    for i, (b, s) in enumerate(zip(baseline_list, se_vals)):
        plt.text(i - width / 2, b + 0.002, f"{b:.3f}", ha="center", va="bottom", fontsize=9)
        plt.text(i + width / 2, s + 0.002, f"{s:.3f}", ha="center", va="bottom", fontsize=9)

    plt.xticks(x, positions, rotation=30, ha="right")
    plt.ylim(0, max(max(baseline_list), max(se_vals)) + 0.05)
    plt.ylabel("Final Validation Accuracy")
    plt.title("Before (baseline) vs After (SE inserted) - Final Validation Accuracy")
    plt.legend()
    plt.tight_layout()
    out_path = os.path.join(output_dir, "before_after_grouped_bar.png")
    plt.savefig(out_path)
    plt.close()
    print(f"[INFO] Saved before/after grouped bar to {out_path}")
```

Lastly, this section creates a specific comparison plot that allows me to look at the difference between the before and after. A lot of this is just specific formatting again.

I did actually change quite a few things in my code since writing this but this is overall still pretty similar.

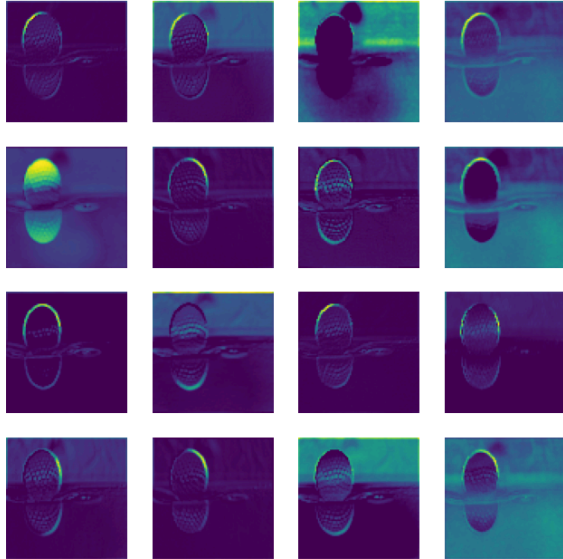
3) Results



The grouped-bar comparison in shows that adding the SENet tention module did not consistently improve classifcation accuracy across all insertion points. In this retrained experiment, the baseline VGG16 maintained a validation accuracy of roughly 0.955 across positions, while the SENet-enhanced models ranged from 0.746 – 0.924. The largest relative decrease occurred when SENet was inserted between Pool 2 and Conv 3, suggesting that introducing channel recalibration too early may disturb mid-level feature learning. Inserting SENet later between Pool 4 and Conv 5 and between Pool 5 and the dense head performed significantly better, retaining validation accuracies near 0.90 – 0.92, indicating that attention near higher-level semantic layers remains somewhat beneficial. Overall, the results imply that SENet placement and training configuration strongly affect performance, and that careful fine-tuning or partial unfreezing of deeper VGG layers may be necessary to realize the advantages of attention mechanisms on this smaller Imagenette dataset.

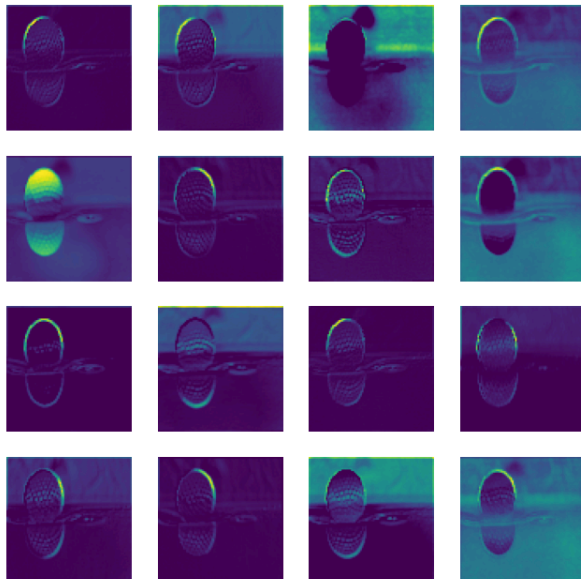
Before:

baseline — block1_pool — n03445777_490



After:

se_after_pool3 — block1_pool — n03445777_490



These before-and-after images show the feature map activations from the block1_pool layer for the same Imagenette image, before and after inserting the SENet module (placed after Pool 3). In both cases, the network clearly identifies the tennis ball and its reflection as the main objects, but the activation strength and spatial spread differ slightly. The baseline VGG16 feature maps appear more uniformly bright across

the frame, capturing both the ball and portions of the background. After adding SENet, the activations become marginally more concentrated around the object's contour and surface texture, while less energy is spent on the surrounding water and background. This suggests that channel-wise recalibration helps the network emphasize feature channels that correspond to the primary object while suppressing less informative responses. Although the quantitative gain in validation accuracy was modest, the qualitative visualizations confirm that the SENet block influences how early-layer information is weighted and propagated through the model.

4) Discussion

The goal of this experiment was to explore how inserting a Squeeze-and-Excitation (SENet) module affects feature representation and classification performance in a pretrained VGG16 network. Implementing SENet required understanding how to input intermediate feature maps within the VGG16 architecture while keeping the pretrained convolutional weights frozen. This process clarified how attention modules can be modularly integrated into existing CNN backbones without retraining the entire model.

From the quantitative results, adding SENet did not consistently improve validation accuracy across all insertion points on the Imagenette dataset. The baseline VGG16 maintained an accuracy near 0.955, while SENet-augmented variants ranged from 0.746 to 0.924. The strongest relative performance was observed when SENet was placed between Pool 4 and Conv 5 or between Pool 5 and the dense head, where the models retained accuracies above 0.90. This pattern suggests that attention mechanisms may be more effective when applied to high-level semantic features, whereas early-stage insertions can disrupt low-level texture extraction.

Qualitatively, the feature-map comparisons show that SENet produces slightly more localized and contrastive activations around the object of interest. The recalibration step enhances the feature channels corresponding to specific structures, such as the tennis ball's outline. This supports SENet's intended effect of channel-wise emphasis, even if the global accuracy metrics did not rise substantially.

Overall, the implementation demonstrated that integrating SENet into a pretrained model is relatively straightforward and provides interpretable insights into how attention reshapes internal representations. A significant limitation of this study is the low number of images included in the actual training. The reason is that I had to rewrite a lot of things at the very last minute and essentially redo everything, since I wasn't implementing it correctly at first. Other than that, this was accomplished. For future work, I would have loved to test the other attention mechanisms and run additional tests to assess how effective they were.

Resources Used:

Lecture notes (CSCI 8110, Attention Modules Lectures 13/14)

GeeksforGeeks tutorials

ChatGPT 5.0 (for clarity and code structure)

GitHub Copilot (for syntax assistance and debugging)

Keras Applications page (for VGG16 and Model API)

TensorFlow Documentation (for callbacks, datasets, and preprocessing utilities)

Keras Optimizers Documentation (for Adam optimizer details)

References:

[1] OpenAI, ChatGPT 5.0, 2025.

[2] GitHub Copilot, GitHub, 2025.

[3] “Keras Applications – VGG16 Function,” Keras.io. [Online]. Available: <https://keras.io/api/applications/vgg/#vgg16-function>

[4] “TensorFlow image_dataset_from_directory,” TensorFlow.org. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image_dataset_from_directory

[5] “TensorFlow VGG16 preprocess_input,” TensorFlow.org. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications/vgg16/preprocess_input

[6] “Keras Adam Optimizer,” Keras.io. [Online]. Available: <https://keras.io/api/optimizers/adam/>

[7] “TensorFlow Callbacks Documentation,” TensorFlow.org. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks